

§ 11.4 *BM算法

11.4.1 思路与框架

■ 构思

KMP算法的思路可概括为：当前比对一旦失配，即利用此前的比对（无论成功或失败）所提供的信息，尽可能长距离地移动模式串。其精妙之处在于，无需显式地反复保存或更新比对的历史，而是独立于具体的主串，事先根据模式串预测出所有可能出现的失配情况，并将这些信息“浓缩”为一张next表。就其总体思路而言，本节将要介绍的BM算法^③与KMP算法类似，二者的区别仅在于预测和利用“历史”信息的具体策略与方法。

BM算法中，模式串P与主串T的对准位置依然“自左向右”推移，而在每一对准位置却是“自右向左”地逐一比对各字符。具体地，在每一轮自右向左的比对过程中，一旦发现失配，则将P右移一定距离并再次与T对准，然后重新一轮自右向左的扫描比对。为实现高效率，BM算法同样需要充分利用以往的比对所提供的信息，使得P可以“安全地”向后移动尽可能远的距离。

■ 主体框架

BM算法的主体框架，可实现如代码11.6所示。

```
1 int match(char* P, char* T) { //Boyer-Morre算法（完全版，兼顾Bad Character与Good Suffix）
2     int* bc = buildBC(P); int* gs = buildGS(P); //构造BC表和GS表
3     size_t i = 0; //模式串相对于主串的起始位置（初始时与主串左对齐）
4     while (strlen(T) >= i + strlen(P)) { //不断右移（距离可能不止一个字符）模式串
5         int j = strlen(P) - 1; //从模式串最末尾的字符开始
6         while (P[j] == T[i + j]) //自右向左比对
7             if (0 > --j) break;
8         if (0 > j) //若极大匹配后缀 == 整个模式串（说明已经完全匹配）
9             break; //返回匹配位置
10        else //否则，适当地移动模式串
11            i += __max(gs[j], j - bc[ T[i + j] ]); //位移量根据BC表和GS表选择大者
12    }
13    delete [] gs; delete [] bc; //销毁GS表和BC表
14    return i;
15 }
```

代码11.6 BM主算法

可见，这里采用了蛮力算法后一版本（310页代码11.2）的方式，借助整数i和j指示主串中当前的对齐位置T[i]和模式串中接受比对的字符P[j]。不过，一旦局部失配，这里不再是机械地令i += 1并在下一字符处重新对齐，而是采用了两种启发式策略确定最大的安全移动距离。为此，需经过预处理，根据模式串P整理出坏字符和好后缀两类信息。

与KMP一样，算法过程中指针i始终单调递增；相应地，P相对于T的位置也绝不后退。

^③ 由R. S. Boyer和J. S. Moore于1977年发明^[61]

11.4.2 坏字符策略

■ 坏字符

如图11.10(a)和(b)所示, 若模式串P当前在主串T中的对齐位置为*i*, 且在这一轮自右向左将P与substr(T, *i*, *m*)的比对过程中, 在P[*j*]处首次发现失配:

$$T[i + j] = 'X' \neq 'Y' = P[j]$$

则将'X'称作坏字符 (bad character)。问题是:

接下来应该选择P中哪个字符对准T[*i* + *j*], 然后开始新一轮自右向左的比对?

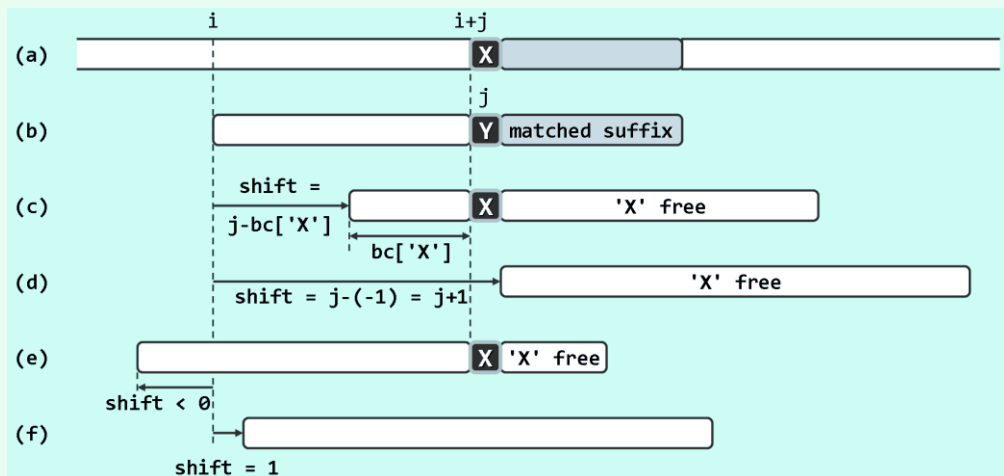


图11.10 坏字符策略：通过右移模式串P，使T[*i* + *j*]重新得到匹配

若P与T的某一（包括T[*i* + *j*]在内的）子串匹配，则必然在T[*i* + *j*] = 'X'处匹配；反之，若与T[*i* + *j*]对准的字符不是'X'，则必然失配。故如图11.10(c)所示，只需找出P中的每一字符'X'，分别与T[*i* + *j*] = 'X'对准，并执行一轮自右向左的扫描比对。不难看出，对应于每个这样的字符'X'，P的位移量仅取决于原失配位置*j*，以及'X'在P中的秩，而与T和*i*无关！

■ BC[]表

若P中包含多个'X'，则是否真地有必要逐一尝试呢？实际上，这既不现实——如此将无法确保主串指针*i*永不回退——更不必要。一种简便而高效的做法是，仅尝试P中最靠右的字符'X'（若存在）。与KMP算法类似，如此便可在确保不致遗漏匹配的前提下，始终单向地滑动模式串。具体如图11.10(c)所示，若P中最靠右的字符'X'为P[*k*] = 'X'，则P的右移量即为*j* - *k*。

同样幸运的是，对于任一给定的模式串P，*k*值只取决于字符T[*i* + *j*] = 'X'，因此可将其视作从字符表到整数（P中字符的秩）的一个函数：

$$bc(c) = \begin{cases} k & (\text{若 } P[k] = c, \text{ 且对所有的 } i > k \text{ 都有 } P[i] \neq c) \\ -1 & (\text{若 } P[] \text{ 中不含字符 } c) \end{cases}$$

故如代码11.6所示，如当前对齐位置为*i*，则一旦出现坏字符P[*j*] = 'Y'，即重新对齐于：

$$i += j - bc[T[i + j]]$$

并启动新一轮比对。为此可仿照KMP算法，预先将函数bc()整理为一份查询表，称作BC表。